

计算机网络第六章—传输层考点总结

1 传输层概述与服务模型

传输层的位置与作用

- 传输层位于应用层和网络层之间，基于网络层提供的服务，向分布式应用程序提供通信服务 [p.4]。
- 按照因特网的“端到端”设计原则：应用程序只运行在终端上，不需要为网络设备编写程序 [p.4]。
- 传输层应提供进程之间本地通信的抽象：运行在不同终端上的应用进程仿佛直接连在一起 [p.4]。
- 引入传输层的原因：消除网络层的多样性和不可靠性；提供从源主机到目的端主机的可靠的、价格合理的、与实际使用的网络无关的信息传输 [p.3]。
- 传输层向上提供标准的传输服务，向下屏蔽不同的通信子网 [p.3]。

传输层的基本功能与服务

- 网络层提供“尽最大努力交付”（best-effort）的服务：不保证交付、不保证按序交付、不保证数据完整、不保证延迟、不保证带宽 [p.6]。
- 传输层的有所为、有所不为：可以通过差错恢复、重排序等手段提供可靠、按序的交付服务；但无法提供延迟保证、带宽保证等服务 [p.6]。
- 传输层的核心功能是实现进程之间的端到端通信 [p.7]。

传输服务的两种类型

- 面向连接的传输服务（**Connection-Oriented**）：包括三个阶段——连接建立、数据传输、释放连接 [p.9]。
- 释放连接的方式有两种：
 - 不对称方式：任何一方都可以关闭双向连接 [p.9]。
 - 对称方式：每个方向的连接单独关闭，双方都执行 Disconnect 才能关闭整条连接（TCP 采用此方式） [p.9]。
- 无连接的传输服务（**Connectionless**）：不需要建立连接，每个报文独立传输 [p.9]。

传输协议数据单元（TPDU）

- 传输层中实现传输层协议的软件称为“传输实体”（Transport Entity） [p.8]。
- 传输层之间传输的报文称为“传输协议数据单元”（TPDU, Transport Protocol Data Unit） [p.8]。

2 套接字 (Socket) 与端口 (Port)

基本概念

- 传输层协议接收到数据后，通过端口号区分数据是发送给哪一个应用程序 [p.13]。
- **端口号 (Port)**: 16 位的标识符，标识传输层协议和应用程序之间的数据接口。端口号由不同主机上的 TCP 和 UDP 协议独立分配，不可能全局唯一 [p.13]。
- **套接字 (Socket)**: 端口号 + IP 地址，唯一标识一端。在 TCP 协议中，一条连接两端的套接字就可以唯一标识该连接 [p.13]。
- 套接字是应用层和传输层的接口，也是应用程序和网络之间的 API [p.5]。

端口号的分类 (IANA 划分)

- **熟知端口号 (Well-Known Ports)**: 范围 0 ~1023，由 IANA 统一分配给标准的 Internet 服务 [p.14-15]。
- **注册端口号 (Registered Ports)**: 范围 1024 ~49151，为用户开发的新网络应用程序在 IANA 登记 [p.15]。
- **临时端口号 (Ephemeral Ports)**: 范围 49152 ~65535，由客户端的 TCP/UDP 软件随机选取，只对一次进程通信有效 [p.15]。

常见熟知端口号

端口号	协议	服务说明
53	UDP	DNS 域名服务 [p.16]
67/68	UDP	DHCP 动态主机配置协议 [p.16]
69	UDP	TFTP 简单文件传送协议 [p.16]
161/162	UDP	SNMP 简单网络管理协议 [p.16]
520	UDP	RIP 路由信息协议 [p.16]
20	TCP	FTP 文件传输协议 (数据连接) [p.17]
21	TCP	FTP 文件传输协议 (控制连接) [p.17]
23	TCP	TELNET 网络虚拟终端协议 [p.17]
25	TCP	SMTP 简单邮件传输协议 [p.17]
80	TCP	HTTP 超文本传输协议 [p.17]
179	TCP	BGP 边界路由协议 [p.17]

五元组标识

五元组是唯一标识一个通信会话的组合: [p.18]

五元组 = (协议, 本地地址, 本地端口号, 远程地址, 远程端口号)

3 连接建立与释放

连接建立面临的问题

- 网络可能丢失、重复分组，特别是延迟重复分组可能干扰新的连接 [p.23]。
- 解决重复分组的措施：
 - (1) 一次性传输地址（非重复的 TSAP）：每次连接赋予新的传输地址，连接释放后废弃 [p.23-24]。缺点：不支持进程服务器（没有公认的众所周知的 TSAP） [p.24]。
 - (2) 连接标识符（过时连接表）：为每个传输连接分配连接标识符存入 TPDU 中，连接释放后将标识加入过时连接表；新连接请求到达时核对该表 [p.25]。缺点：需要保留历史记录，机器崩溃后丢失过时连接表 [p.25]。
 - (3) 分组生命期控制（TTL）：将分组的生命期控制在一定范围内 [p.23]。

三次握手（Three-Way Handshake）

三次握手方案解决了由于网络层会丢失、存储和重复分组带来的问题，不要求通信双方同步序列号，可以识别旧的重复请求 [p.26, 28]。

正常建立连接的过程 [p.26-27]：

1. A 发出序号为 x 的建立连接请求 CR TPDU。
2. B 发出序号为 y 的接受连接确认 ACK TPDU，并确认 A 的序号为 x 的建立连接请求。
3. A 发出序号为 x 的第一个数据 DATA TPDU，并确认 B 的序号为 y 的接受连接确认。

用 TCP 术语表示 [p.49]：

1. CLOSED → SYN-SENT: <SEQ=100><CTL=SYN> →
2. LISTEN → SYN-RCVD: ← <SEQ=300><ACK=101><CTL=SYN,ACK> ←
3. SYN-SENT → ESTABLISHED: → <SEQ=101><ACK=301><CTL=ACK> →

连接释放

- 连接释放需要避免数据丢失 [p.29]。
- TCP 采用四次挥手（对称释放方式）：每个方向单独关闭，双方都执行断开操作才能完全关闭连接 [p.53]。

基于时钟的方法

解决了重复数据 TPDU 延迟到达的问题，但无法区分建立连接请求是否是当前的 [p.28]。

4 差错控制与流量控制

滑动窗口协议（Sliding Window Protocol）

- 传输层使用滑动窗口协议进行差错控制和流量控制 [p.31]。

- 较大的滑动窗口可以实现更高的传输效率 [p.31]。
- TCP 采用滑动窗口机制完成数据段的可信传输 [p.50]。

多路复用与逆向多路复用

- 多路复用（**Multiplexing**）：多个传输层连接共用一个网络连接 [p.32]。
- 逆向多路复用（**Inverse Multiplexing**）：有多条网络连接，把一个连接上的流量分摊到多条网络路径上 [p.32]。

Crash Recovery（崩溃恢复）

- 服务器可能发生 3 种事件：确认（A）、将数据写入输出进程（W）、崩溃（C） [p.33]。
- S0 状态—没有未确认的段；S1 状态—发出一个段但尚未确认 [p.33]。
- 没有一种策略能够在所有情况下都能正确恢复 [p.33]。

5 TCP 协议（传输控制协议）

TCP 与 UDP 概述

- TCP 和 UDP 是 TCP/IP 协议簇中的传输层协议，处于 IP 协议与应用层协议之间 [p.34]。
- **TCP**：弥补了 IP 协议不可靠的特性，向应用层提供可靠的面向连接的服务，实现了通信进程的连接与释放、滑动窗口协议及拥塞控制等功能 [p.34]。
- **UDP**：实现的是简单的无连接不可靠数据报传输服务，完成了应用进程的识别，不提供流控或拥塞管理机制 [p.34]。

TCP 的主要特点

- 支持面向连接的传输服务 [p.37]
- 支持字节流（**Byte Stream**）的传输—发送端应用程序提交的字节流被 TCP 分割成报文段发送，接收端重新组装 [p.37-38]
- 支持全双工通信（**Full-Duplex**） [p.37]
- 支持同时建立多个并发的 TCP 连接 [p.37]
- 支持可靠的传输服务 [p.37]

TCP 段格式（TCP Segment Format）

TCP 段的结构（固定头部 20 字节，可扩展到 60 字节） [p.41-42]：

字段	说明
源端口号（Source Port）	16 位 [p.43]
目的端口号（Dest Port）	16 位 [p.43]
序号（Sequence Number）	32 位，以字节为单位，标识本报文段第一个数据字节的编号 [p.43]
确认号（ACK Number）	32 位，标识期望收到的下一个字节序号 [p.43]
首部长度（Header Len）	4 位，以 4 字节为单位表示 TCP 首部长度 [p.43]
保留（Reserved）	4 位（实际 6 位，含后来添加的 2 位） [p.43]

TCP 标志位（Flag Bits, 8 位）

- **URG**：紧急数据标志。URG=1 表示包含紧急数据；与紧急指针配合使用，紧急指针指向段内最后一个紧急数据字节的位置。即使窗口大小为 0 也可以发送紧急数据 [p.44]。
- **ACK**：确认标志。ACK=1 表示确认号字段有效 [p.44]。在连接建立后的所有报文段中 ACK 都必须置 1。
- **PSH**：推送标志。PSH=1 表示请求远端 TCP 立即将本报文段传送给其应用层，不要等待缓冲区填满 [p.44]。
- **RST**：复位标志。RST=1 表示出现严重错误（如主机崩溃），必须释放连接然后重建；也可用于拒绝非法数据段或连接请求 [p.44]。
- **SYN**：同步标志。SYN=1, ACK=0 表示连接请求；SYN=1, ACK=1 表示连接响应 [p.44]。
- **FIN**：终止标志。FIN=1 表示发送方已没有数据发送，用于释放连接 [p.44]。
- **CWR**：拥塞窗口减小（Congestion Window Reduced）。发送端设置 CWR 信号，表示已减慢发送速度 [p.45]。
- **ECE**：ECN-Echo。接收端收到网络层拥塞指示后设置 ECE，提示发送方拥塞 [p.45]。

TCP 其他重要字段

- **窗口大小（Window Size）**：16 位，目的机告诉源主机在确认了的字节后还可以发送多少个字节（即接收窗口 rwnd） [p.45]。
- **校验和（Checksum）**：16 位，计算时包括 TCP 包头和数据以及一个伪包头（Pseudo-Header） [p.45-46]。
- **紧急指针（Urgent Pointer）**：16 位，只在 URG 标志设置时才有效 [p.45]。
- **选项域（Options）**：可变长度，用于参数协商（如 MSS 最大段大小等） [p.45]。

TCP 的连接建立、数据传输与释放管理

连接建立（三次握手） [p.49]：

1. 客户端发送 SYN=1, seq=x 的报文段（SYN-SENT 状态）
2. 服务器回复 SYN=1, ACK=1, seq=y, ack=x+1（SYN-RCVD 状态）

3. 客户端回复 ACK=1, seq=x+1, ack=y+1（双方进入 ESTABLISHED 状态）

注：SYN 报文段消耗一个序号，因此确认号为收到的序号 +1。

数据传输（滑动窗口机制）[p.50]:

- TCP 采用滑动窗口机制实现数据段的可信传输 [p.50]。
- 发送窗口大小 = $\min(\text{cwnd}, \text{rwnd})$ [p.64]
- 其中 cwnd 为拥塞窗口（Congestion Window），rwnd 为接收方通告的窗口。

连接释放（四次挥手）[p.53]:

1. 主动关闭方发送 FIN=1, seq=u（FIN-WAIT-1 状态）
2. 被动关闭方回复 ACK=1, ack=u+1（CLOSE-WAIT 状态，半关闭）
3. 被动关闭方发送 FIN=1, seq=v, ack=u+1（LAST-ACK 状态）
4. 主动关闭方回复 ACK=1, ack=v+1，进入 TIME-WAIT 状态

TCP 定时器管理

- **重发定时器（Retransmission Timer）**: 每次发送数据段时启动，如果在超时前被确认则关闭，超时则重传数据段并重新计时。也间接用于管理网络拥塞 [p.54-55]。
- **持续定时器（Persistence Timer）**: 用于防止死锁。当发送方收到窗口为 0 的应答后启动，超时后发送探测数据段获取当前窗口大小 [p.59]。
- **保活定时器（Keepalive Timer）**: 当连接长时间闲置时，检测对方是否仍然存在，若未得到响应则终止连接 [p.60]。
- **TIME-WAIT 定时器（连接定时器）**: 用于断开连接操作的 TIME-WAIT 状态，设置为分组最长生命期的两倍（2MSL），确保连接断开后分组完全消失 [p.61]。

RTT 估计与超时值的计算

- **SampleRTT**: 测量从发出某个报文段到收到其确认报文段之间经过的时间 [p.57]。
- **EstimatedRTT**: 通过指数加权移动平均（EWMA）估算平均 RTT [p.57]:

$$\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}$$

通常 $\alpha = 0.125$ 。

- **DevRTT**: 估算 SampleRTT 与 EstimatedRTT 的偏差 [p.58]:

$$\text{DevRTT} = (1 - \beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|$$

通常 $\beta = 0.25$ 。

- **超时值（TimeoutInterval）** [p.58]:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$

- 合理设置超时值的重要性：太小容易产生不必要的重传；太大则丢包恢复时间太长 [p.57]。

提高传输效率的策略

- **Nagle 算法**: 在发送方缓存应用程序的数据, 等到形成一个较大的段后再发出 [p.52]。
- **延迟确认 (Delayed ACK)**: 接收方延迟发送确认报文, 以减少 ACK 数量 [p.52]。
- **Clark 算法**: 解决糊涂窗口综合征 (Silly Window Syndrome, SWS) ——只有当有比较大的接收窗口时才告诉发送方 [p.52]。

零窗口处理

- 当接收方发生缓冲区溢出时, 发送窗口为 0 的应答通知发送方等待 [p.51, 59]。
- **窗口探测 (Window Probe)**:
 - (1) 发送紧急数据 [p.51]
 - (2) 发送 1 字节段, 强制接收端重新宣告下一个期望的字节和窗口大小 [p.51]

6 TCP 拥塞控制 (Congestion Control)

基本概念

- 在端到端的连接中, 导致网络 congestion 的两个潜在因素是**网络能力**和**接收能力** [p.64]。
- 当网络快而接收方缓存小时, congestion 发生在接收方主机处 [p.64]。
- 当接收方具有足够的缓存而网络较慢时, congestion 发生在网络中的某个地方 [p.64]。
- TCP 发送方维护两个窗口 [p.64]:
 - **可变发送窗口**: 由接收方的缓存能力决定 (即 `rwnd`)。
 - **拥塞窗口 (`cwnd`)**: 依照拥塞控制算法变化。
- 实际发送窗口大小取两者的最小值: $\text{EffectiveWindow} = \min(\text{rwnd}, \text{cwnd})$ 。

慢启动算法 (Slow Start)

- 连接建立时, `cwnd` 初始化为 1 个 MSS (最大报文段长度) [p.62]。
- 每收到一个 ACK, `cwnd` 增加 1 个 MSS (即每个 RTT `cwnd` 翻倍) [p.62, 65]。
- 增长方式为**指数增长**: `cwnd` 从 1 开始, 经历 1 个 RTT 变为 2, 再变为 4, 8, 16……[p.65]。
- 当 `cwnd` 达到慢启动阈值 (`ssthresh`) 时, 转为拥塞避免算法。
- 在 TCP Tahoe 中, 当发生超时时: $\text{ssthresh} = \text{cwnd} / 2$, `cwnd` = 1, 重新开始慢启动。

拥塞避免算法 (Congestion Avoidance)

- 当 $\text{cwnd} \geq \text{ssthresh}$ 时, 进入拥塞避免阶段 [p.62]。
- 每个 RTT, cwnd 增加 1 个 MSS (线性增长 / 加法增大, Additive Increase) [p.65]。
- 拥塞窗口随往返次数的典型变化序列: 1, 2, 4, 8, 16, 17, 18, 19, 20, 21, 22, 23, 12, 13, 14, 15... [p.67]
- 发生超时时处理 (TCP Tahoe): $\text{ssthresh} = \text{cwnd} / 2$, $\text{cwnd} = 1$, 重新进入慢启动 [p.65]。

快速重传 (Fast Retransmit)

- 当发送方连续收到 **3 个重复 ACK** (即共 4 个相同 ACK 号的 ACK) 时, 认为该报文段已丢失 [p.65]。
- 发送方立即重传丢失的报文段, 而不必等待重传定时器超时。
- 快速重传机制可以减少因超时等待造成的时间浪费。

快速恢复 (Fast Recovery)

- 发生快速重传时 (收到 3 个重复 ACK), 执行快速恢复而非慢启动 [p.65]。
- **TCP Reno 的处理方式:**
 1. $\text{ssthresh} = \text{cwnd} / 2$ (乘法减小, Multiplicative Decrease)
 2. $\text{cwnd} = \text{ssthresh} + 3 \times \text{MSS}$ (不直接降到 1)
 3. 此后每收到一个重复 ACK, $\text{cwnd} += \text{MSS}$
 4. 收到新的 ACK 后, $\text{cwnd} = \text{ssthresh}$, 进入拥塞避免阶段
- **TCP Tahoe vs TCP Reno:**
 - TCP Tahoe: 超时或 3 个重复 ACK 都触发同样的处理—— $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$, 重新慢启动。
 - TCP Reno: 超时触发慢启动 ($\text{cwnd} = 1$); 3 个重复 ACK 触发快速恢复 ($\text{cwnd} = \text{ssthresh} + 3$), 之后进入拥塞避免。Reno 比 Tahoe 效率更高。

AIMD 原则 (Additive Increase / Multiplicative Decrease)

- **加法增大 (AI):** 在没有拥塞时, 每个 RTT 将 cwnd 线性增加 (拥塞避免阶段) [p.65]。
- **乘法减小 (MD):** 检测到拥塞时, 将 cwnd 减半 (拥塞发生时乘以因子 0.5) [p.65]。
- 这是 TCP 拥塞控制的核心设计原则, 已被证明可以使多个 TCP 连接公平共享带宽。

7 UDP 协议（用户数据报协议）

UDP 的特点

- UDP 是一种无连接的、不可靠的传输层协议 [p.70]。
- 不保证数据完整到达目的地，没有流量和拥塞控制机制 [p.68]。
- UDP 保留应用程序定义的报文边界：既不合并也不拆分应用报文 [p.68]。
- UDP 是小的、节约资源的传输层协议，操作执行比 TCP 快 [p.68]。
- 提供了与 IP 协议的接口，通过端口号复用多个进程；可选的端到端错误检测功能 [p.68]。
- 如果检测到收到的分组出错，UDP 丢弃该分组，既不确认，也不通知发送端和请求重传 [p.70]。
- 只要应用能将报文尽可能快地发送（无连接建立延迟，不进行拥塞控制和流量控制），报头开销小 [p.76]。

UDP 段格式

- UDP 数据报包括 8 字节的头部和数据部分 [p.69, 74]。
- 最大长度 65515 字节（理论值）[p.74]。

字段	大小	说明
源端口号（Source Port）	16 位	发送方端口号 [p.74]
目的端口号（Dest Port）	16 位	接收方端口号 [p.74]
消息长度（Length）	16 位	UDP 数据报总长度（头部 + 数据）[p.74]
校验和（Checksum）	16 位	可选，校验 UDP 头部 + 数据 + 伪头部 [p.74]

UDP 校验和与伪头部

- UDP 校验和包含了 IPv4 伪头部（Pseudo-Header）[p.73, 75]。
- 伪头部包含源 IP 地址、目的 IP 地址、协议号、UDP 长度等字段 [p.75]。
- 校验和计算方法：先将校验和字段置 0，对 UDP 头部 + 数据 + 伪头部按 16 位分组，反码求和，结果取反码即为校验和 [p.73]。

UDP 的适用场景

- 容忍丢包但对延迟敏感的应用：如流媒体（视频/音频）[p.76]。
- 以单次请求/响应为主的应用：如 DNS [p.76]。
- 若应用要求基于 UDP 进行可靠传输：由应用层自己实现可靠性 [p.76]。

8 核心考点速记

关键公式

1. **EstimatedRTT**（指数加权移动平均）：

$$\text{EstimatedRTT} = (1 - \alpha) \times \text{EstimatedRTT} + \alpha \times \text{SampleRTT}, \quad \alpha = 0.125$$

2. **DevRTT**（RTT 偏差）：

$$\text{DevRTT} = (1 - \beta) \times \text{DevRTT} + \beta \times |\text{SampleRTT} - \text{EstimatedRTT}|, \quad \beta = 0.25$$

3. **TimeoutInterval**（超时值）：

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \times \text{DevRTT}$$

4. 有效窗口：

$$\text{EffectiveWindow} = \min(\text{rwnd}, \text{cwnd})$$

5. **AIMD**：

- 加法增大（AI）：拥塞避免阶段每个 RTT 将 cwnd 线性加 1 个 MSS
- 乘法减小（MD）：检测到拥塞时 cwnd 减半（ $\text{cwnd} = \text{cwnd}/2$ ）

6. 拥塞控制状态转换：

- 慢启动阶段：cwnd 指数增长（每 RTT 翻倍）， $\text{cwnd} < \text{ssthresh}$
- 拥塞避免阶段：cwnd 线性增长， $\text{cwnd} \geq \text{ssthresh}$
- 超时： $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1$ ，回到慢启动（Tahoe & Reno）
- 3 个重复 ACK（快速重传）： $\text{ssthresh} = \text{cwnd}/2$; Tahoe: $\text{cwnd}=1$ 回慢启动; Reno: $\text{cwnd}=\text{ssthresh}+3$ 进快速恢复后转拥塞避免

核心概念

1. 传输层作用：提供进程间端到端通信，屏蔽网络层不可靠性 [p.3-4]
2. 端口号范围：熟知 0–1023，注册 1024–49151，临时 49152–65535 [p.14-15]
3. 套接字 = IP 地址 + 端口号 [p.13]
4. 五元组 = (协议, 本地 IP, 本地端口, 远程 IP, 远程端口) [p.18]
5. 三次握手： $\text{SYN} \rightarrow \text{SYN}+\text{ACK} \rightarrow \text{ACK}$ [p.26-27, 49]
6. 四次挥手： $\text{FIN} \rightarrow \text{ACK} \rightarrow \text{FIN} \rightarrow \text{ACK}$ [p.53]
7. **TCP 标志位**：URG, ACK, PSH, RST, SYN, FIN; CWR, ECE（拥塞控制）[p.44-45]
8. 滑动窗口：差错控制 + 流量控制的实现机制 [p.31]

9. 发送窗口 = $\min(\text{cwnd}, \text{rwnd})$ [p.64]
10. **TCP 四种定时器**: 重发定时器、持续定时器、保活定时器、TIME-WAIT 定时器 [p.54-61]
11. **Nagle 算法**: 缓存小数据形成大段再发送, 减少小报文开销 [p.52]
12. **糊涂窗口综合征 (SWS)**: Clark 算法解决, 仅有较大接收窗口才通告发送方 [p.52]
13. **慢启动**: cwnd 初始 1 MSS, 每 RTT 翻倍 (指数增长) [p.62, 65]
14. **拥塞避免**: $\text{cwnd} \geq \text{ssthresh}$ 后, 每 RTT 线性增加 1 MSS [p.62]
15. **快速重传**: 收到 3 个重复 ACK 立即重传, 不等待超时 [p.65]
16. **快速恢复**: Reno 特有, 3 个重复 ACK 后 $\text{cwnd} = \text{ssthresh} + 3$, 不降到 1 [p.65]
17. **Tahoe vs Reno**: Tahoe 在超时或 3 个重复 ACK 后都 $\text{cwnd}=1$ 重新慢启动; Reno 在 3 个重复 ACK 后执行快速恢复 [p.65]
18. **TCP 面向连接, 全双工, 字节流, 可靠; UDP 无连接, 不可靠, 报报文边界, 高效** [p.34, 37, 68]
19. **UDP 头部仅 8 字节**: 源端口 (2) + 目的端口 (2) + 长度 (2) + 校验和 (2) [p.69]
20. **TCP 头部最小 20 字节**: 可扩展到 60 字节 [p.42]
21. **TCP 校验和包含伪头部** (源 IP + 目的 IP + 协议号 + TCP 长度) [p.45-46]
22. **UDP 适用场景**: DNS、流媒体、TFTP、SNMP 等对延迟敏感或仅需简单请求/响应的应用 [p.76]
23. **TCP/IP 体系中——UDP 对应 IP 的无连接服务; TCP 弥补 IP 的不可靠性, 提供可靠面向连接服务** [p.34]